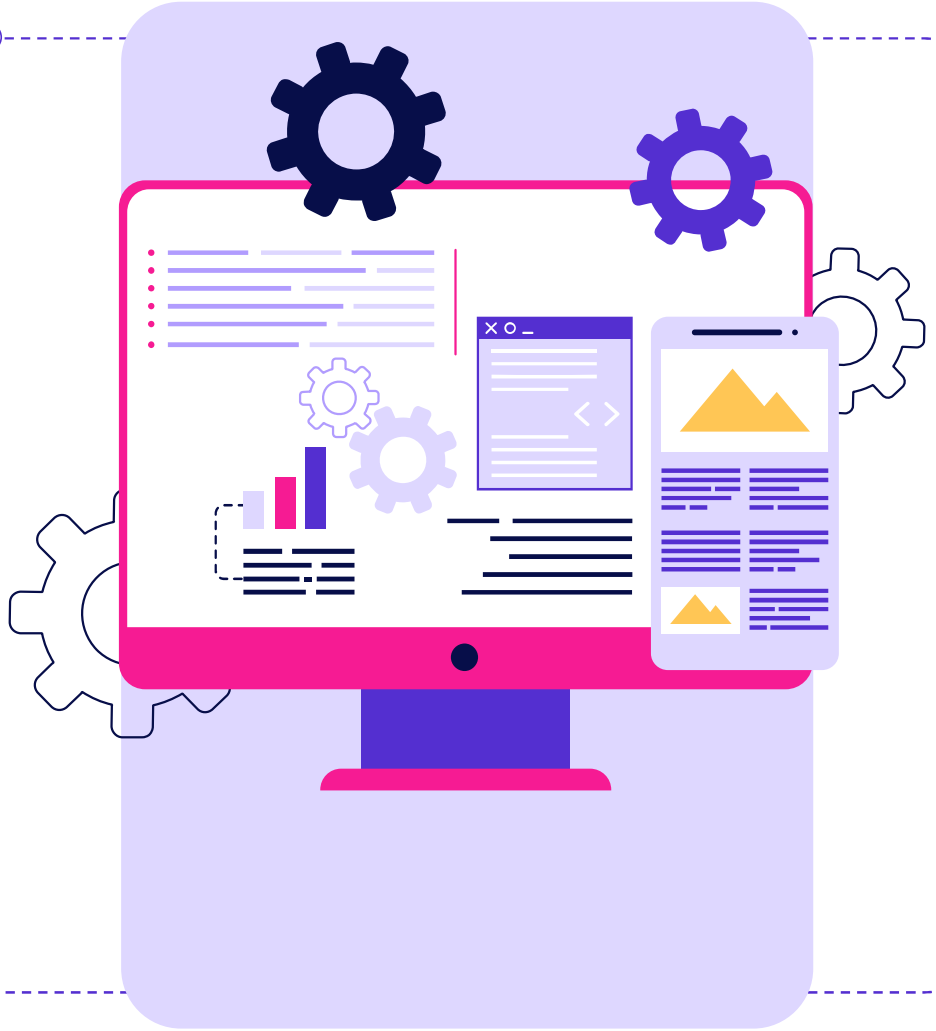
Estrutura de Dados 2



GRAFOS

Profª Juliana Franciscani

Roteiro

01 Introdução

02 Definição

03 Aplicações

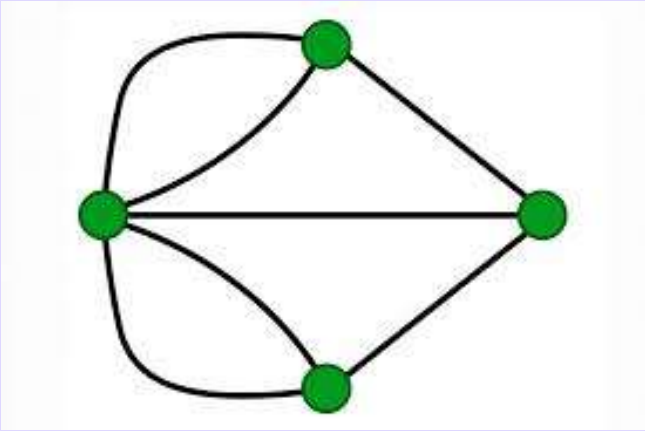
04 Nomenclaturas

05 Tipos de Grafos

06 Exercícios

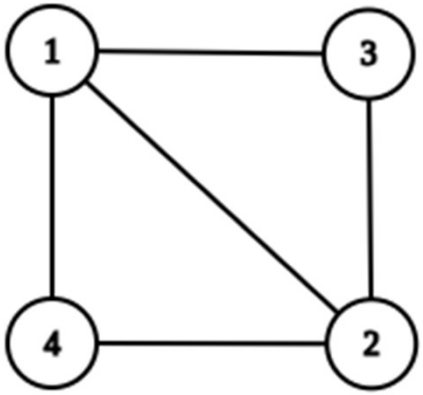
Introdução a Grafos

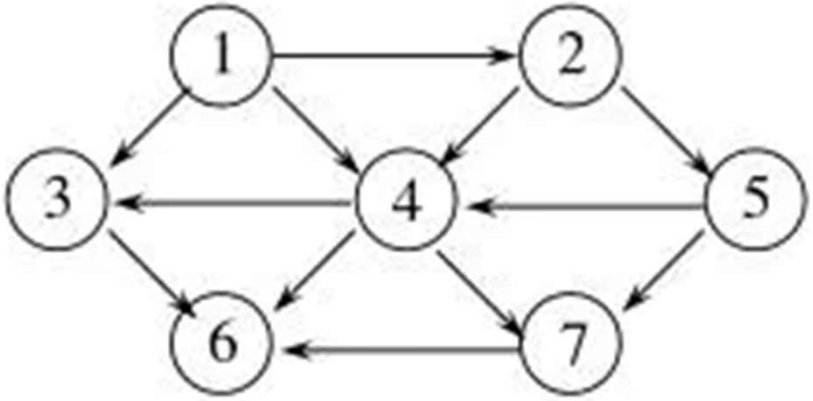
- Teoria dos Grafos:
  - Euler em 1736
  - Resolução de um problema de travessia de pontes de Konigsberg
  - Deu origem a teoria dos grafos: descobriu que não havia solução possível para o problema



O que é um grafo?

- É um conjunto de vértices (nós) e arestas (arcos)
- Cada aresta liga dois vértices
- Representação de modelos matemáticos que representam relações entre objetos



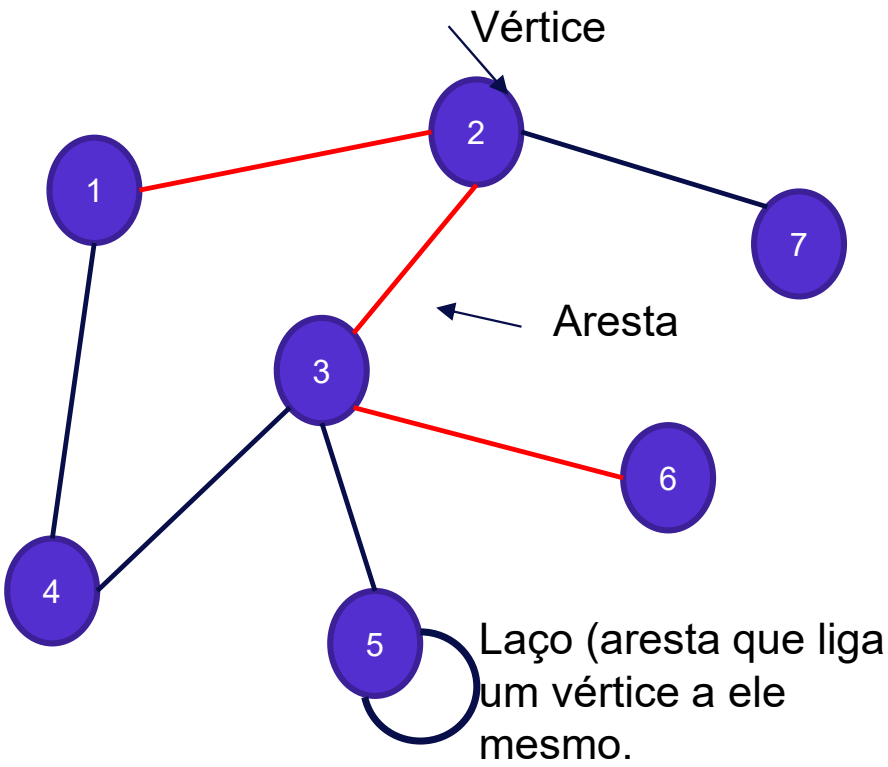


Aplicações

- Comércio: marketing, relação entre produtos
- Transporte: logística, fluxo, coleta de lixo.
- Biologia: genética, relações
- Medicina: relação entre pessoas, epidemia, mapeamento na psicologia
- Sistema de Transporte: metrô, trem, ônibus (malha)
- Tabela de jogos, jogadas e movimentos no xadrez
- Redes de computadores
- Internet: redes sociais, árvore genealógica/hierarquia, links páginas
- Compiladores: autômatos

Nomenclatura

- **Grafos são formados por:**
  - Conjunto não vazio de Vértices V:
    - entidades/objetos representadas no grafo
  - Conjunto de Arestas A
    - Relação entre dois vértices
- **Caminhos:** Sequência de vértices de modo que existe sempre uma aresta ligando o vértice anterior com o seguinte.
- **Caminho Simples:** Nenhum dos vértices do caminho se repete.
- **Comprimento do caminho:** É o número de arestas que o caminho usa.

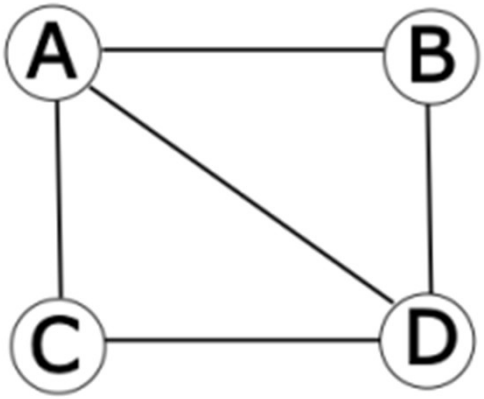


G (V,A)
V = {1,2,3,4,5,6,7}
A = {{1,2}, {1,4}, {2,3}, {2,7}, {3,4}, {3,5}, {3,6},{5,5}}

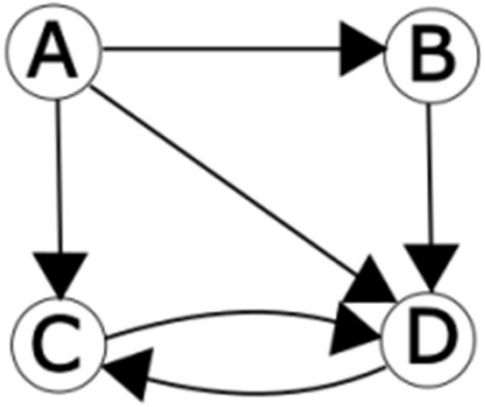
Tipos de Grafos

- Grafos não direcionados
  - Não há direcionamento
  - Grau: número de arestas que entram e saem de um vértice

- Grafos Direcionados ou Dígrafos
  - Orientação quanto ao sentido da aresta.
  - Grau de entrada
  - Grau de saída



Grau:
G(A): 3
G(B): 2
G(C): 2
G(D): 3



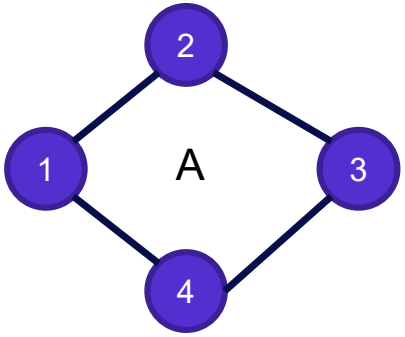
Grau de Entrada:
G(A): 0
G(B): 1
G(C): 2
G(D): 3

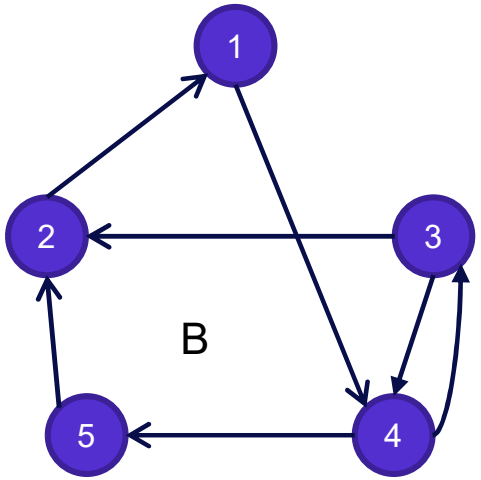
Grau de Saída:
G(A): 3
G(B): 1
G(C): 1
G(D): 1

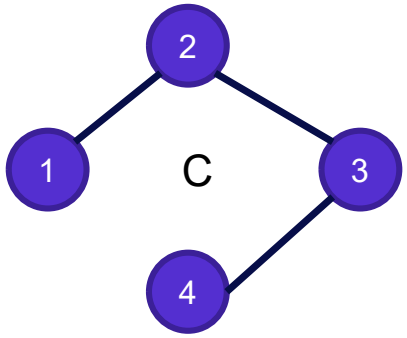
Quando houver Laço, para o Grau conta-se 2 vezes em grafos não direcionados.
E nos grafos direcionados contará 1 vez como entrada e outra como saída.

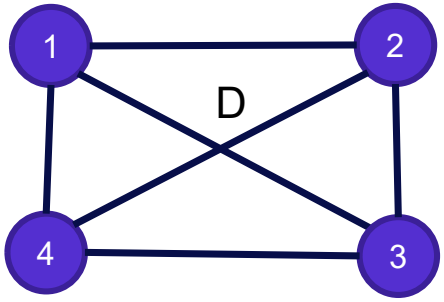
Tipos de Grafos

- **Grafo Trivial**
  - Formado por um único vértice
- **Grafo Simples**
  - Não direcionado, sem laços e sem arestas paralelas.
- **Grafo Completo**
  - Grafo simples onde cada vértice se conecta a todos os outros vértices.
- **Grafo Regular:**
  - todos os vértices possuem o mesmo grau.
- **Grafo Cíclico**
  - Contém ciclo: caminho que começa e termina sempre no mesmo vértice.
- **Grafo Acíclico**
  - Não contém ciclo
- **Grafo com Múltiplas Arestas**
  - Permite mais de uma aresta conectando o mesmo par de vértices (Arestas Paralelas)
- **Grafo Bipartido**
  - Dois conjuntos de vértices. Vértices do mesmo conjunto não são ligados

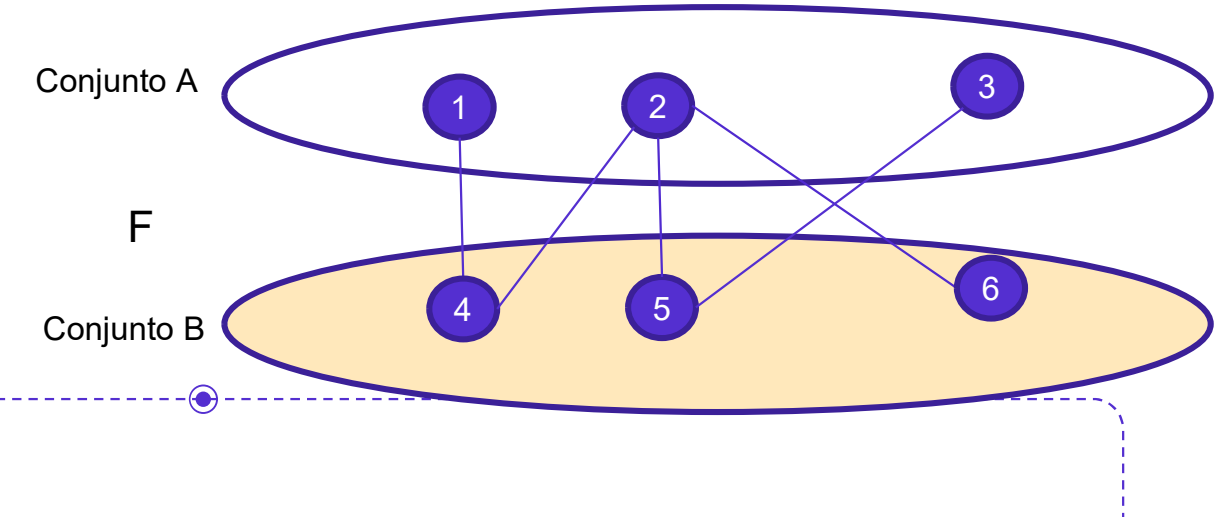

1
2
3
4
A

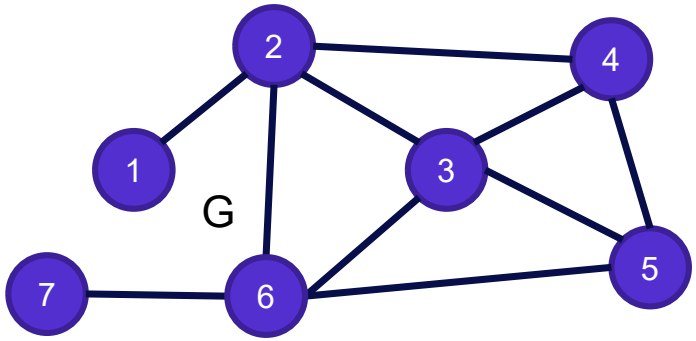

1
2
3
4
5
B


1
2
3
4
C


1
2
3
4
D

1
E

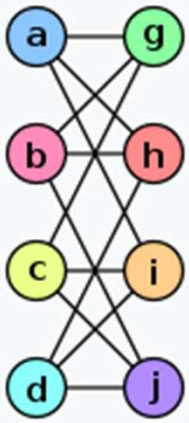
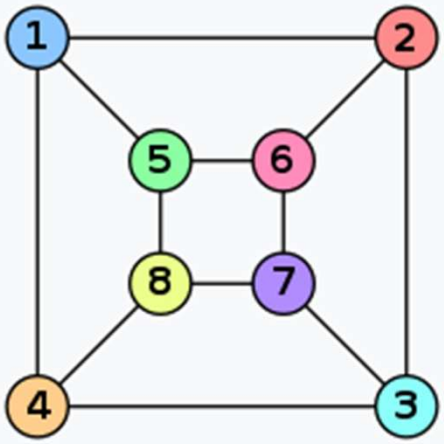

Conjunto A
1
2
3
F
4
5
6
Conjunto B


1
2
3
4
5
6
7
G

Tipo de Grafo

- **Isomorfo:**

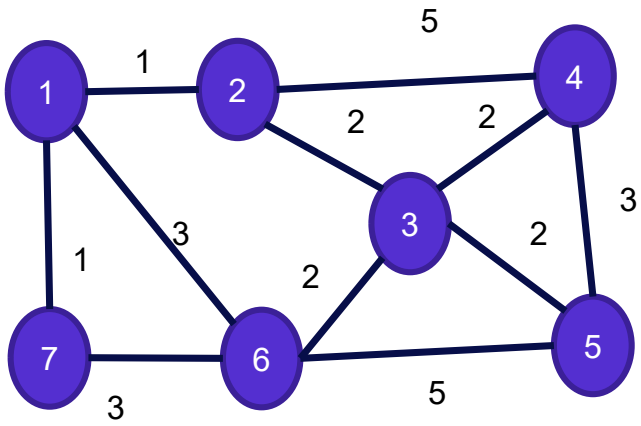
Dois grafos são chamados de isomorfos se existe uma função que faça o mapeamento de vértices e arestas de modo que o dois grafos se tornem coincidentes.

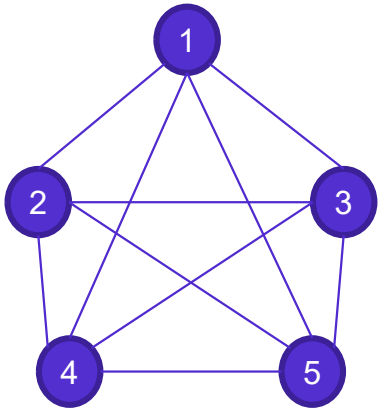
Grafo G	Grafo H	Um isomorfismo entre G e H
		$f(a) = 1$ $f(b) = 6$ $f(c) = 8$ $f(d) = 3$ $f(g) = 5$ $f(h) = 2$ $f(i) = 4$ $f(j) = 7$

Tipos de Grafos

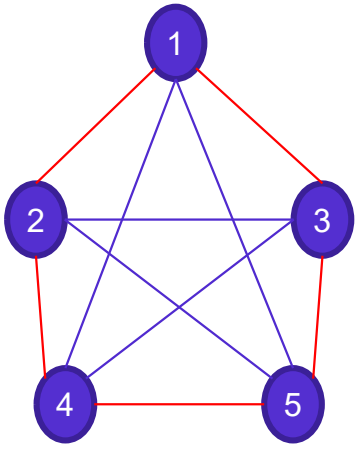
- **Ponderado:**
Um grafo que possui peso associado a cada uma das arestas

- **Hamiltoniano:**
Um grafo que possui um caminho que visita cada vértice apenas uma vez

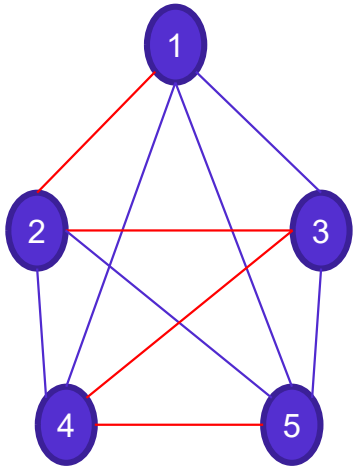




Grafo



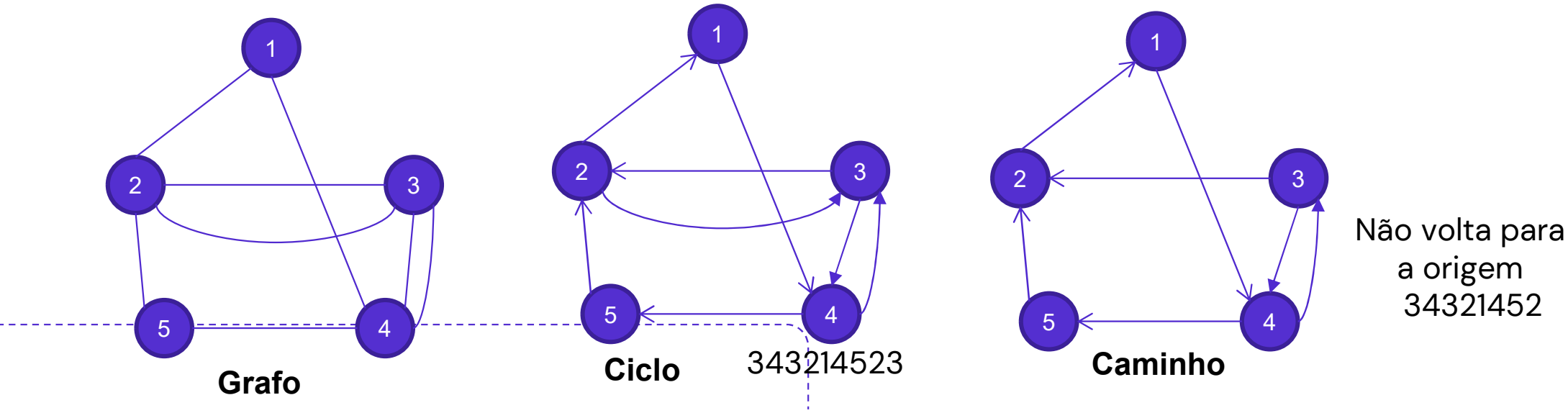
Ciclo



Caminho

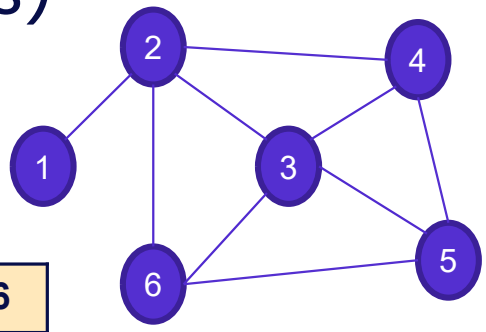
Tipos de Grafos

- **Grafo Eureliano**
  - Possui um ciclo que visita cada aresta apenas uma vez.
  - Pode passar por um vértice mais de uma vez, buscando passar por todas as arestas.
  - O grafo deve ser direcionado.



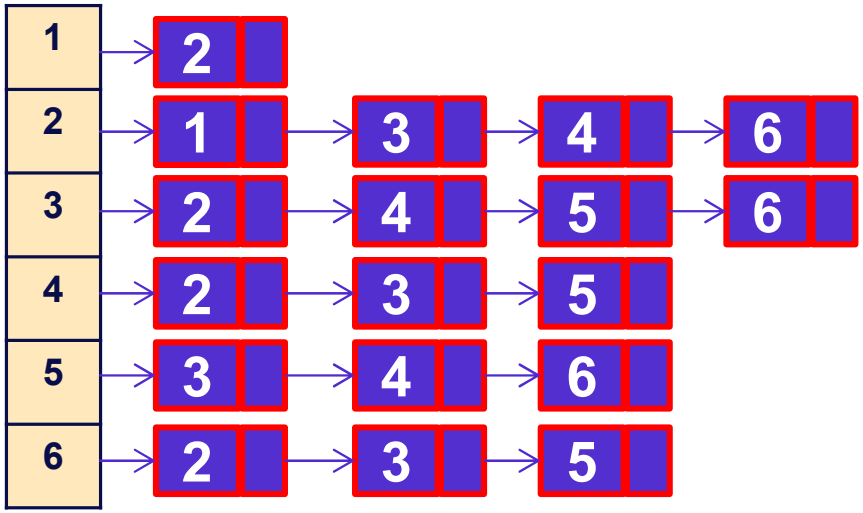
Representação de Grafo

- Matriz de Adjacência (muitas ligações)



(i,j)	1	2	3	4	5	6
1	0	1	0	0	0	0
2	1	0	1	1	0	1
3	0	1	0	1	1	1
4	0	1	1	0	1	0
5	0	0	1	1	0	1
6	0	1	1	0	1	0

- Lista de Adjacência (poucas ligações)



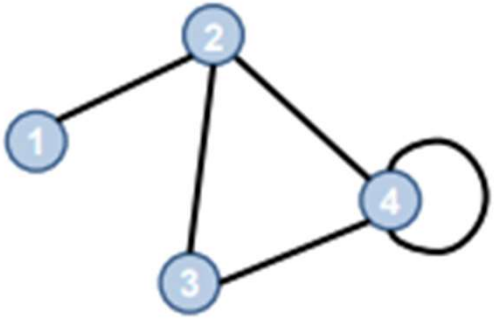
Representação de Grafo – Matriz de Adjacência

- Utiliza uma matriz N x N para armazenar o grafo, onde N é o número de vértices
  - Alto custo computacional, $O(N^2)$
- Uma aresta é representada por uma marca na posição (i , j) da matriz
  - Aresta liga o vértice i ao j

Representação de Grafo

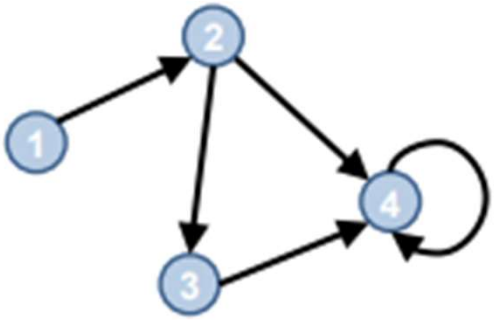
- Matriz de adjacência

GRAFO



	1	2	3	4
1	0	1	0	0
2	1	0	1	1
3	0	1	0	1
4	0	1	1	1

DIGRAFO



	1	2	3	4
1	0	1	0	0
2	0	0	1	1
3	0	0	0	1
4	0	0	0	1

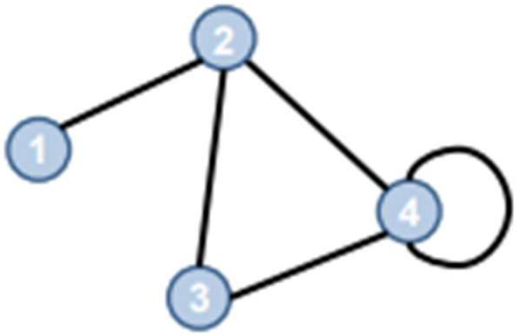
Representação de Grafo – Lista de Adjacência

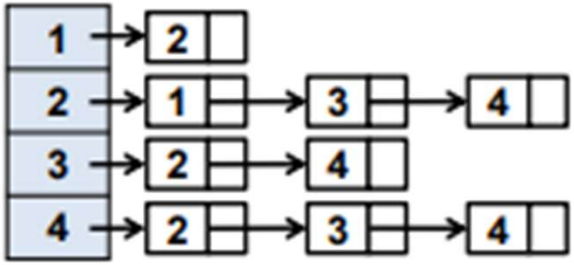
- Utiliza uma lista de vértices para descrever as relações entre os vértices.
  - Um grafo contendo N vértices utiliza um array de ponteiros de tamanho N para armazenar os vértices do grafo
  - Para cada vértice é criada uma lista de arestas, onde cada posição da lista armazena o índice do vértice a qual aquele vértice se conecta

Representação de Grafo

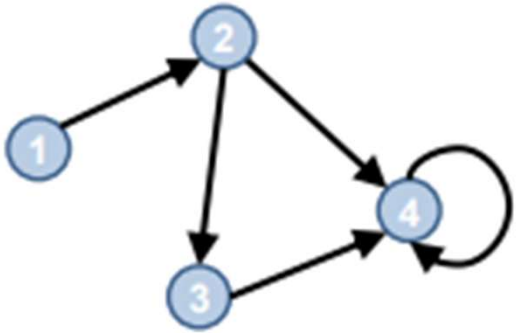
- Lista de adjacência

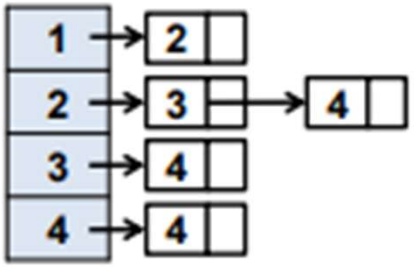
GRAFO



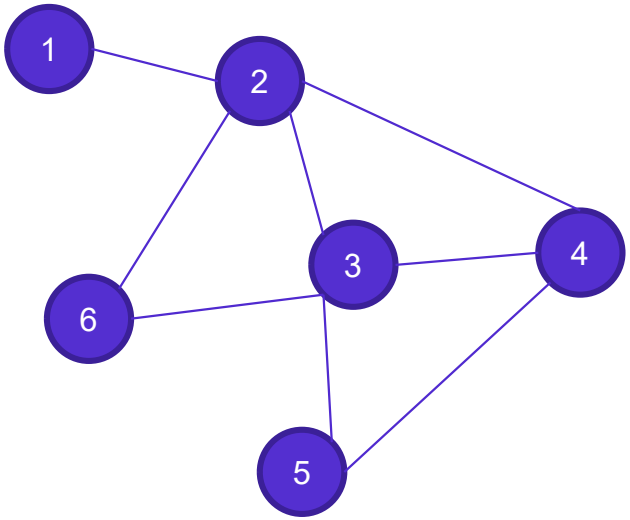


DIGRAFO

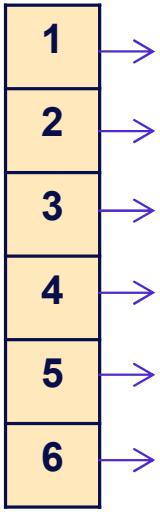




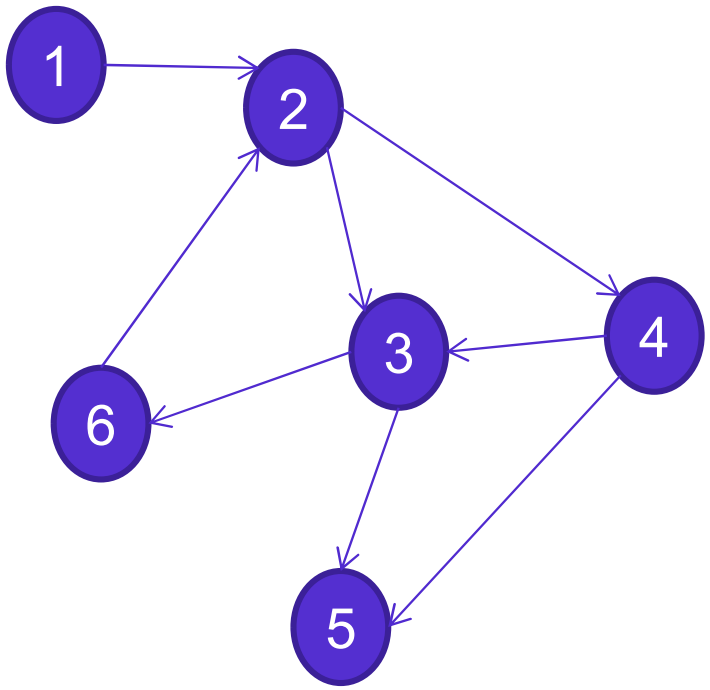
Exercícios



	1	2	3	4	5	6
1						
2						
3						
4						
5						
6						



Exercício



	1	2	3	4	5	6
1						
2						
3						
4						
5						
6						

1
2
3
4
5
6

Exercícios

$$A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ \\ \end{array} \begin{array}{c} \begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{vmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \end{vmatrix} \end{array}$$

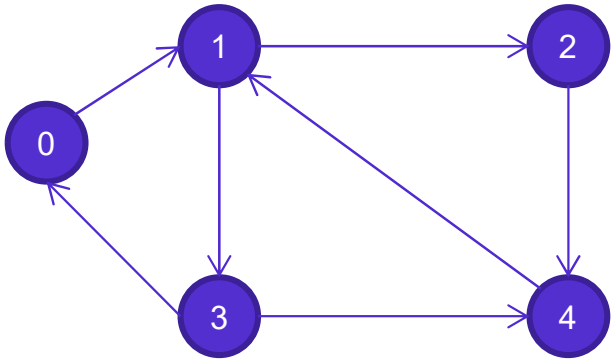
Qual representação utilizar?

- Lista de adjacência é mais indicada para um grafo que possui muitos vértices mas poucas arestas ligando esses vértices.

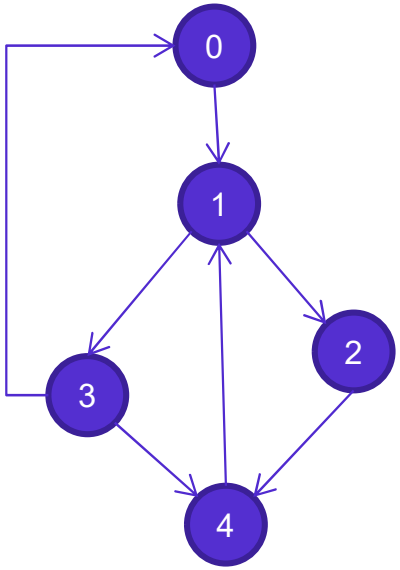
- A medida que o número de arestas cresce e não havendo nenhuma outra informação associada a aresta (por exemplo, seu peso), o uso de uma matriz de adjacência se torna mais eficiente

Busca em Grafos - Profundidade

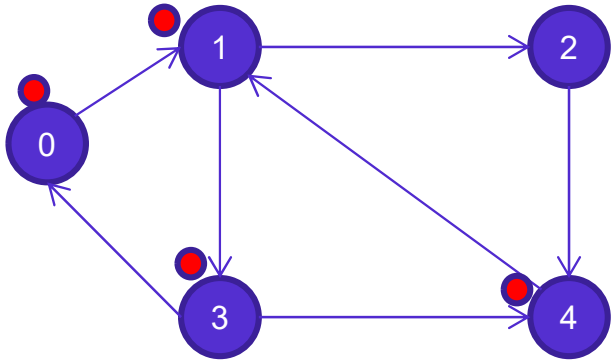
- Partindo de um vértice inicial, explora-se o máximo possível cada um de seus ramos (vizinhos adj) antes de voltar para um novo vértice.



- Usado para:
  - Encontrar componentes conectados e fortemente conectados
  - Ordenação Topológica de um grafo
  - Resolver labirinto (quebra–cabeça)



Busca em Profundidade



Contador = 1 2 3 4

0	0	1	1	1	1	1
1	0	0	2	2	2	2
2	0	0	0	0	0	3
3	0	0	0	3	3	3
4	0	0	0	0	4	4

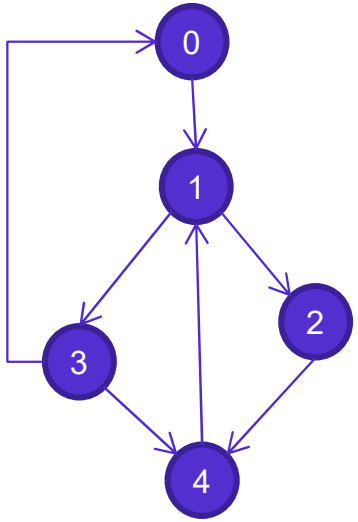
Começa a busca com o vértice 0;
Inicia contador com 1;
Marca o vértice 0 como visitado;
Vai para o próximo adjacente não visitado;
Incrementa contador.

Faz a busca no vértice 1;
Marca o vértice 1 como visitado;
Vai para o próximo adjacente não visitado;
Incrementa contador.

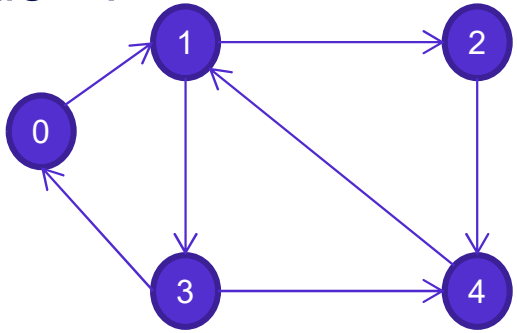
Faz a busca no vértice 3;
Marca o vértice 3 como visitado;
Vai para o próximo adjacente não visitado;
Incrementa contador.

Faz a busca no vértice 4;
Marca o vértice 4 como visitado;
Vai para o próximo adjacente não visitado, como não tem, chama a função de forma recursiva.

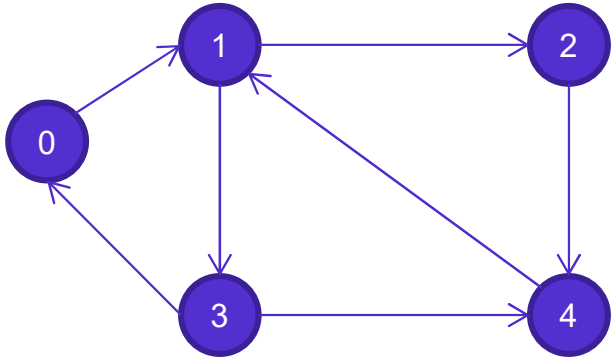
Busca em Grafos - Largura



- Partindo de um vértice inicial, explora-se cada um dos vizinhos adjacentes.
- Em seguida, repete o processo para cada vértice adjacente, visitando os nós ainda não explorado.
- Usado para:
  - Achar componentes conectados
  - Achar todos os nós conectados a apenas 1 componente
  - Encontrar o menor caminho
  - Testar bipartição



Busca em Largura



		Fila		Fila		Fila		Fila	
0	1	0	1	1	1	3	2	1	2	4
1	0		2		2			2	
2	0		0		3			3	
3	0		0		3			3	
4	0		0		0			4	

Inicia a busca com o vértice 0;
Coloca o 0 na fila e marca como visitado;

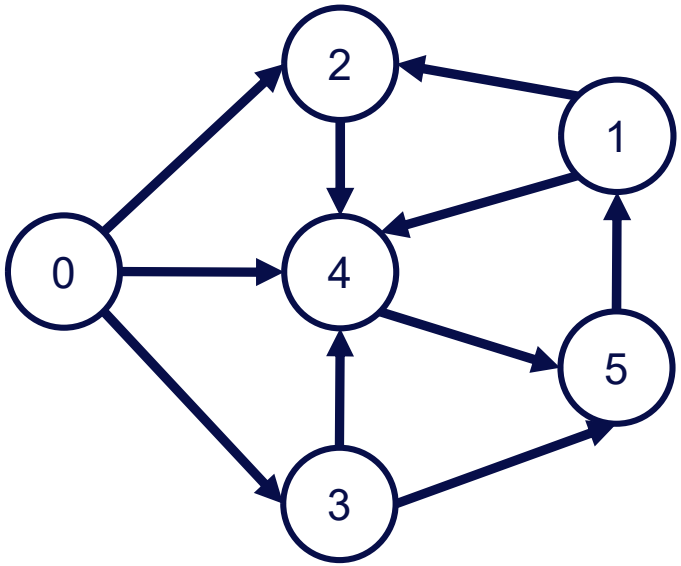
Visita os vizinhos do elemento da fila;
Marca como visitado;
Remove o vértice 0 da fila;
Coloca o vizinho na fila;

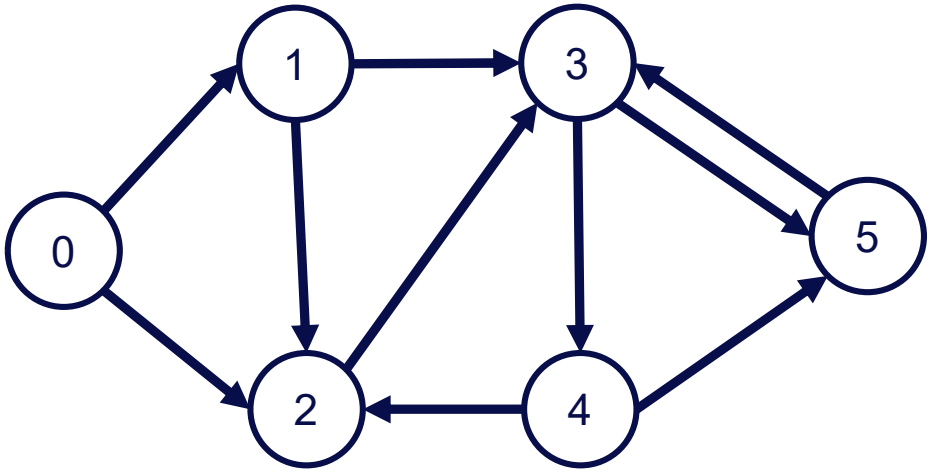
Visita os vizinhos do elemento da fila;
Marca como visitado;
Remove o vértice 1 da fila;
Coloca os vizinhos na fila;

Visita o vizinho do elemento da fila;
Marca como visitado;
Remove o vértice 3 da fila;
Coloca o vizinho na fila;

Exercício:

Busca em Profundidade e Largura dos dois grafos





Implementação Grafo com Lista de Adjacência

- Adota-se o uso de lista de adjacência estática
- Grafos não ponderados
- Funções:
  - CriarGrafo()
  - LiberarGrafo()
  - ImprimirGrafo()
  - InserirAresta()
  - RemoverAresta()
  - BuscaLarguraGrafo()
  - BuscaProfundidadeGrafo()

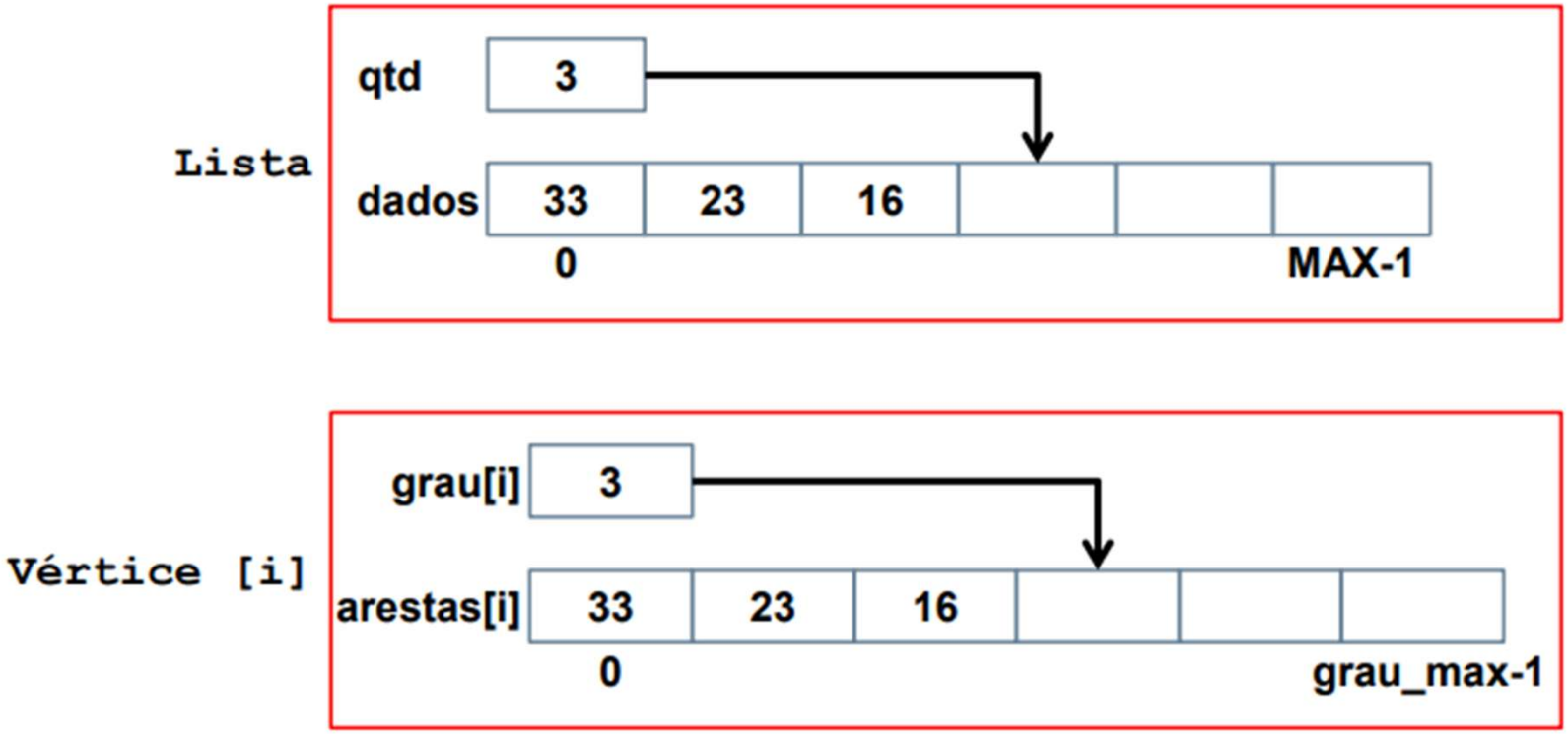
Estrutura do grafo e funções utilizadas

```
struct grafo{
    int numNos;
    int grauMax;
    int** listaArestas;
    int* grauNo;
};
typedef struct grafo Grafo;
```

```
Grafo* CriarGrafo(int numNos, int grauMax);
void LiberarGrafo(Grafo* gr);
int InserirAresta(Grafo* gr, int orig, int dest, int grafoDigrafo);
int RemoverAresta(Grafo* gr, int orig, int dest, int grafoDigrafo);
void BuscaProfundidadeGrafo(Grafo *gr, int ini, int *visitado);
void BuscaLarguraGrafo(Grafo *gr, int ini, int *visitado);
void ImprimirGrafo(Grafo *gr);
```

Funcionamento da lista de Adjacência

- Cada vértice funciona como uma lista estática



main()

```
int main(){
    int grafoDigrafo = 1;
    int nN = 6, gM=5;
    Grafo* gr = CriarGrafo(nN, gM);

    InserirAresta(gr, 0, 2, grafoDigrafo);    InserirAresta(gr, 0, 3, grafoDigrafo);
    InserirAresta(gr, 0, 4, grafoDigrafo);    InserirAresta(gr, 1, 2, grafoDigrafo);
    InserirAresta(gr, 1, 4, grafoDigrafo);    InserirAresta(gr, 2, 4, grafoDigrafo);
    InserirAresta(gr, 3, 4, grafoDigrafo);    InserirAresta(gr, 3, 5, grafoDigrafo);
    InserirAresta(gr, 4, 5, grafoDigrafo);    InserirAresta(gr, 5, 1, grafoDigrafo);

    printf("\nExibe grafo:\n");
    ImprimirGrafo(gr);
    int vis[nN];
    printf("\n\nBusca em Profundidade:\n");
    BuscaProfundidadeGrafo(gr, 0, vis);
    printf("\n\nBusca em Largura:\n");
    BuscaLarguraGrafo(gr, 0, vis);

    LiberarGrafo(gr);
    return 0;
}
```

CriarGrafo()

```
Grafo* CriarGrafo(int numNos, int grauMax){
    Grafo *gr;
    gr = (Grafo*) malloc(sizeof(struct grafo));
    if(gr != NULL){
        int i;
        gr->numNos = numNos;
        gr->grauMax = grauMax;
        gr->grauNo = (int*) calloc(numNos,sizeof(int));
        gr->listaArestas = (int**) malloc(numNos * sizeof(int*));
        for(i=0; i<numNos; i++)
            gr->listaArestas[i] = (int*) malloc(grauMax * sizeof(int));
    }
    return gr;
}
```

InserirAresta()

```
int InserirAresta(Grafo* gr, int orig, int dest,
                  int grafoDigrafo){
    if(gr == NULL) return 0;
    if(orig < 0 || orig >= gr->numNos) return 0;
    if(dest < 0 || dest >= gr->numNos) return 0;

    gr->listaArestas[orig][gr->grauNo[orig]] = dest;
    gr->grauNo[orig]++;

    if(grafoDigrafo == 0)
        InserirAresta(gr,dest,orig,1);
    return 1;
}
```

ImprimirGrafo()

```
void ImprimirGrafo(Grafo *gr){
    if(gr == NULL)
        return;

    int i, j;
    for(i=0; i < gr->numNos; i++){
        printf("%d: ", i);
        for(j=0; j < gr->grauNo[i]; j++)
            printf("%d, ", gr->listaArestas[i][j]);
        printf("\n");
    }
}
```

LiberarGrafo()

```
void LiberarGrafo(Grafo* gr){
    if(gr != NULL){
        int i;
        for(i=0; i<gr->numNos; i++)
            free(gr->listaArestas[i]);
        free(gr->listaArestas);
        free(gr->grauNo);
        free(gr);
    }
}
```

RemoverAresta()

```
int RemoverAresta(Grafo* gr, int orig, int dest, int grafoDigrafo){
    if(gr == NULL) return 0;
    if(orig < 0 || orig >= gr->numNos) return 0;
    if(dest < 0 || dest >= gr->numNos) return 0;
    int i = 0;
    while(i<gr->grauNo[orig] && gr->listaArestas[orig][i] != dest)
        i++;
    if(i == gr->grauNo[orig])//elemento nao encontrado
        return 0;
    gr->grauNo[orig]--;
    gr->listaArestas[orig][i] = gr->listaArestas[orig][gr->grauNo[orig]];
    if(grafoDigrafo == 0)
        RemoverAresta(gr,dest,orig,1);
    return 1;
}
```

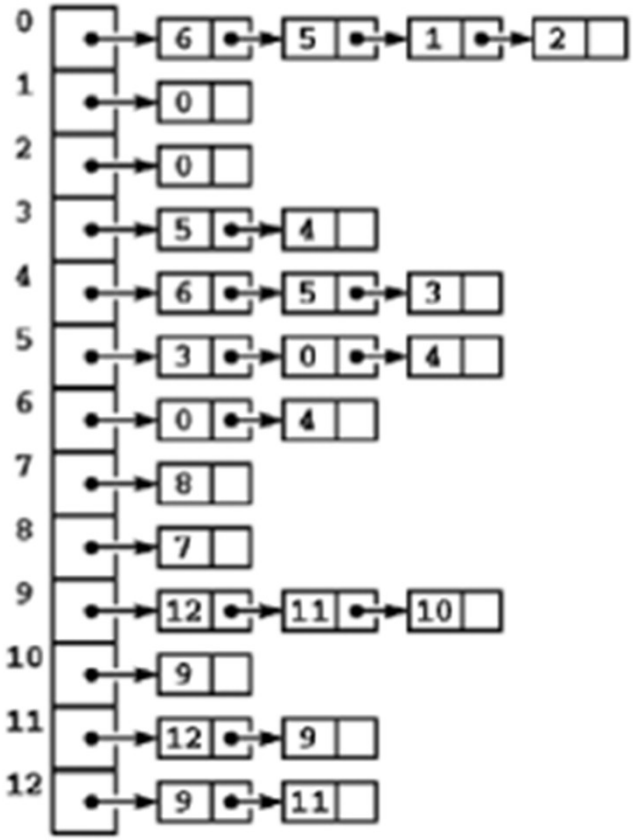
Busca em Profundidade

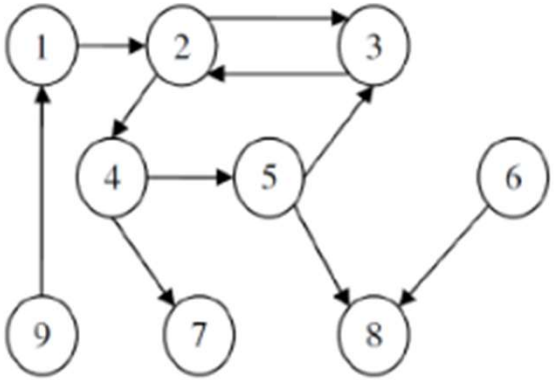
```
void BuscaProfundidade(Grafo *gr, int ini, int *visitado, int cont){
    int i;
    visitado[ini] = cont;
    for(i=0; i<gr->grauNo[ini]; i++){
        if(!visitado[gr->listaArestas[ini][i]])
            BuscaProfundidade(gr,gr->listaArestas[ini][i],visitado,cont+1);
    }
}
void BuscaProfundidadeGrafo(Grafo *gr, int ini, int *visitado){
    int i, cont = 1;
    for(i=0; i<gr->numNos; i++)
        visitado[i] = 0;
    BuscaProfundidade(gr,ini,visitado,cont);
    for(i=0; i < gr->numNos; i++)
        printf("%d -> %d\n",i,visitado[i]);
}
```

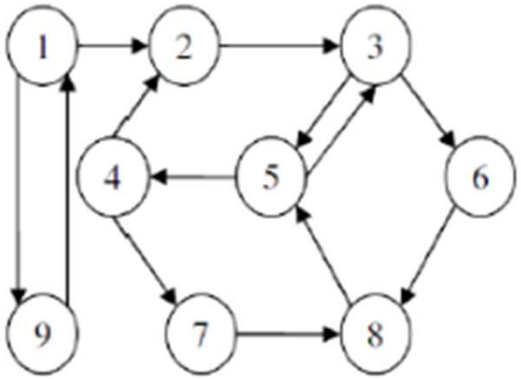
```
void BuscaLarguraGrafo(Grafo *gr, int ini, int *visitado){
    int i, no, numNos, cont = 1, entrou=0,*fila, inicioFila=0,fimFila=0;
    for(i=0; i<gr->numNos; i++) visitado[i] = 0;
    numNos = gr->numNos;
    fila = (int*) malloc(numNos * sizeof(int));
    fila[fimFila] = ini;
    fimFila++;
    visitado[ini] = cont;
    while(inicioFila != fimFila){
        no = fila[inicioFila];
        inicioFila++;
        cont++;
        for(i=0; i<gr->grauNo[no]; i++){
            if(!visitado[gr->listaArestas[no][i]]){
                fila[fimFila] = gr->listaArestas[no][i];
                fimFila++;
                visitado[gr->listaArestas[no][i]] = cont;
                entrou =1;
            }
        }
        if(entrou!=1) cont--;
        entrou =0;
    }
    free(fila);
    for(i=0; i < gr->numNos; i++) printf("%d -> %d\n",i,visitado[i]);
```

Busca em Largura

Exercícios (lousa)







Profundidade

Começa a busca com o vértice 0;
Inicia contador com 1;
Marca o vértice 0 como visitado;
Vai para o próximo adjacente não visitado;
Incrementa contador.

Faz a busca no vértice 1;
Marca o vértice 1 como visitado;
Vai para o próximo adjacente não visitado;
Incrementa contador.

Faz a busca no vértice 3;
Marca o vértice 3 como visitado;
Vai para o próximo adjacente não visitado;
Incrementa contador.

Faz a busca no vértice 4;
Marca o vértice 4 como visitado;
Vai para o próximo adjacente não visitado,
como não tem, chama a função de forma
recursiva.

Largura

Inicia a busca com o vértice 0;
Coloca o 0 na fila e marca como visitado;

Visita os vizinhos do elemento da fila;
Marca como visitado;
Remove o vértice 0 da fila;
Coloca o vizinho na fila;

Visita os vizinhos do elemento da fila;
Marca como visitado;
Remove o vértice 1 da fila;
Coloca os vizinhos na fila;

Visita o vizinho do elemento da fila;
Marca como visitado;
Remove o vértice 3 da fila;
Coloca o vizinho na fila;

Referências

Estrutura de Dados usando C. Aaron M. Tenenbaum, Yedidyah Langsam e Moshe J. Augenstein. Editora Pearson.

Estrutura de Dados e seus Algoritmos. Jayme Luiz Szwarcfiter, Lilian Markenzon. Editora LTC.

Introdução a Estrutura de Dados. Waldemar Celes, Renato Cerqueira e José Lucas Rangel. Editora Campus.

Algoritmos teoria e prática. Thomas H. Cormem, Charles E. Leisserson, Ronald L. Rivet, Ckiffrod Stein. Editora Elsevier.